



CS & IT ENGINEERING



Algorithms

Divide & Conquer

Lecture No.- 01



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Sorting

↳ Insertion

↳ Radix

PYQs

Topics to be Covered



Topic

Topic

Topic

Divide & Conquer



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

TTTe



Topic : Time Complexity

#Q. for ($i = 2; i \leq n; i = i^2$)

for ($j = 1; j \leq n; j++$)

for ($k = 1; k \leq n; k = k + j$)

$x = y + z;$

V. Imp

$\rightarrow O(\log \log n)$

$\rightarrow O(n \log n)$

nested dependent

Overall TC = $O(n \log n * \log(\log n))$

A

$O(\log(\log n))$

B

$O(n \log n)$

C

$O(\log(n \log))$

D

$O(\log(\log n) * n \log n)$

$$\begin{array}{ccccccc}
 i=2 & \rightarrow & 2^2 & \rightarrow & (2^2)^2 & \rightarrow & (2^{2 \times 2})^2 & \dots & \dots & \xrightarrow{\text{Kth iter}} & (2^{2^K}) \\
 & & \downarrow & & \downarrow & & \downarrow & & & & \downarrow \\
 & & 2^{2^1} & & 2^{2^2} & & 2^{2 \times 2 \times 2} & & & & 2^{2^K} \\
 & & & & \downarrow & & \downarrow & & & & \\
 & & & & 2^{2^3} & & & & & &
 \end{array}$$

$$2^{2^K} = n$$

$$2^K = \log n$$

$$K = \log(\log n)$$

j & k combined

$$\Rightarrow \frac{n}{1} + \frac{n}{2} + \frac{n}{3} \dots 1$$

$$= n \left[1 + \frac{1}{2} + \frac{1}{3} \dots \frac{1}{\sqrt{n}} \right]$$

$\underbrace{\hspace{10em}}_{\log n}$

$\Rightarrow$

$$\boxed{n \log n}$$

$$\sum_{i=1}^n \frac{1}{i} = \underline{\underline{\log n}}$$



Topic : Time Complexity



V.V. tricky

#Q. for $i \leftarrow 1$ to $(n - 1)$

└ for $j \leftarrow i + 1$ to n

└ for $k \leftarrow 1$ to j

└ print "*")

nested dependent

What is the number of times "*" gets printed ?

Design Strategies

Algo →

Design & Analysis



Topic : Design Strategies

1. Divide & Conquer (D & q) ✓
2. greedy
3. Dynamic programming → (DP)



Topic : Design Strategies

- When the problem becomes large/complex, then divide the problem into subproblems, into further subproblems, until the subproblem becomes small.
- Solve the smaller problem, combine their results if required, to get the solution of original problem.
- In general a problem is said to be small, if it can be solved in one/two basic operations.

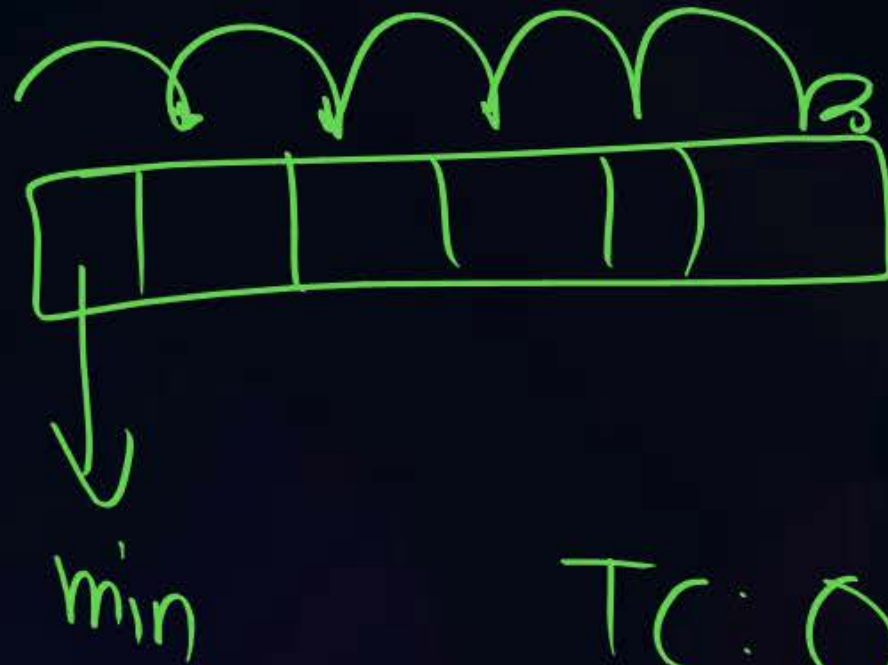
↙
Constant time



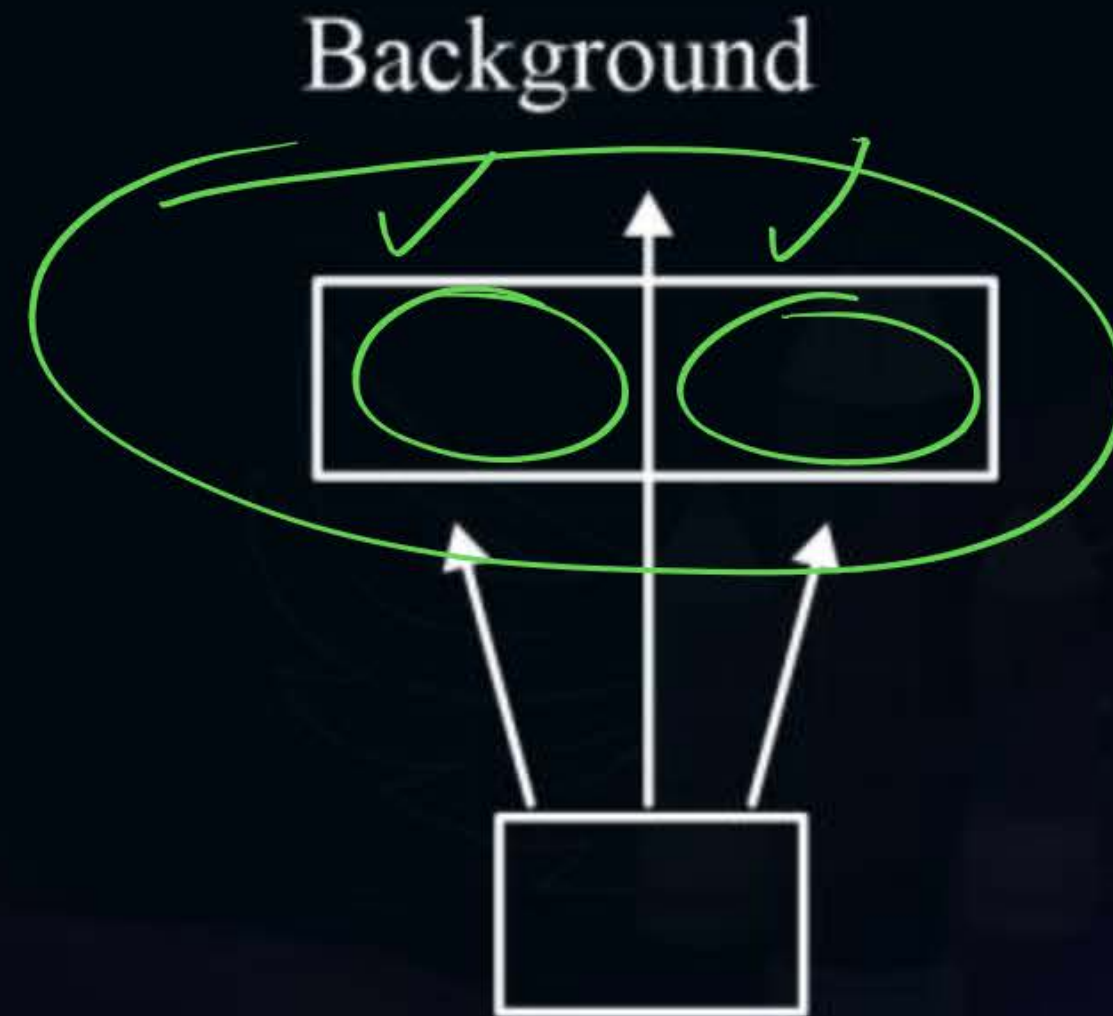
Topic : Design Strategies

Britisher Rule

Eg.2. Divide & Conquer Rule



$$Tc: O(n)$$



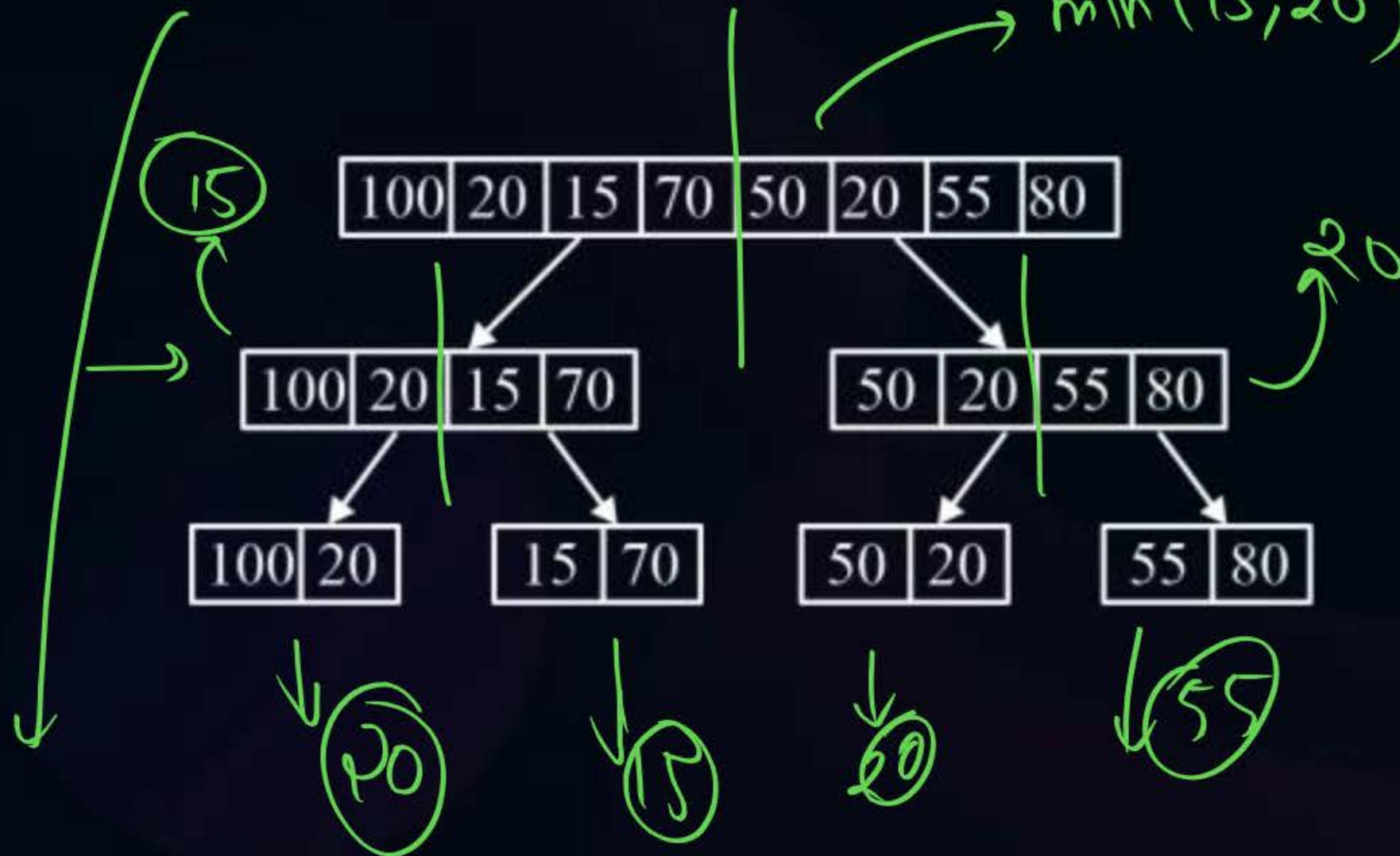


Topic : Design Strategies

#Q. Given an array of elen, find the min element.

Idea

$$\min(15, 20) = 15$$





Topic : Design Strategies

Idea

```
1. Algorithm DAndC(P)
2. {
3.   if Small(P) then return S(P);
4.   else Soln(P)
5.   {
6.     divide P into smaller instances  $P_1, P_2, \dots, P_k, k \geq 1$ ;
7.     Apply DAndC to each of these subproblems;
8.     return Combine(DAndC(P1), DAndC(P2), ..., DAndC(Pk));
9.   }
10. }
```

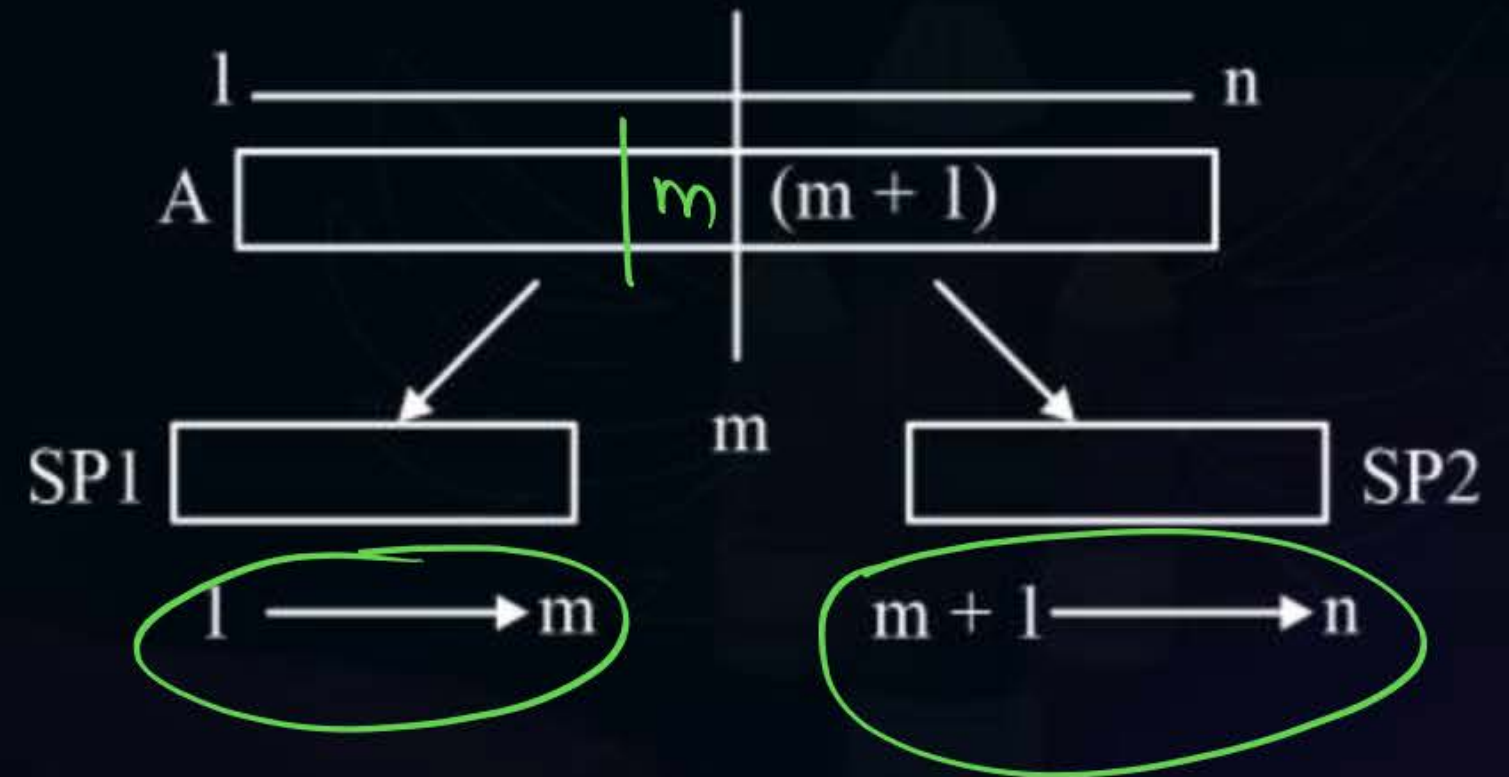



Topic : Design Strategies

Algorithm DandC (A, 1, n)

```
{  
    if (SMALL (1, n))  
        return (S(A, 1, n));  
    else  
    {  
        m ← DIVIDE (1, n)  
        s1 ← DandC (A, 1, m)  
        S2 ← DandC (A, m + 1, n)  
        Combine (S1, S2);  
    }  
}
```

→ overall soln





Topic : Design Strategies

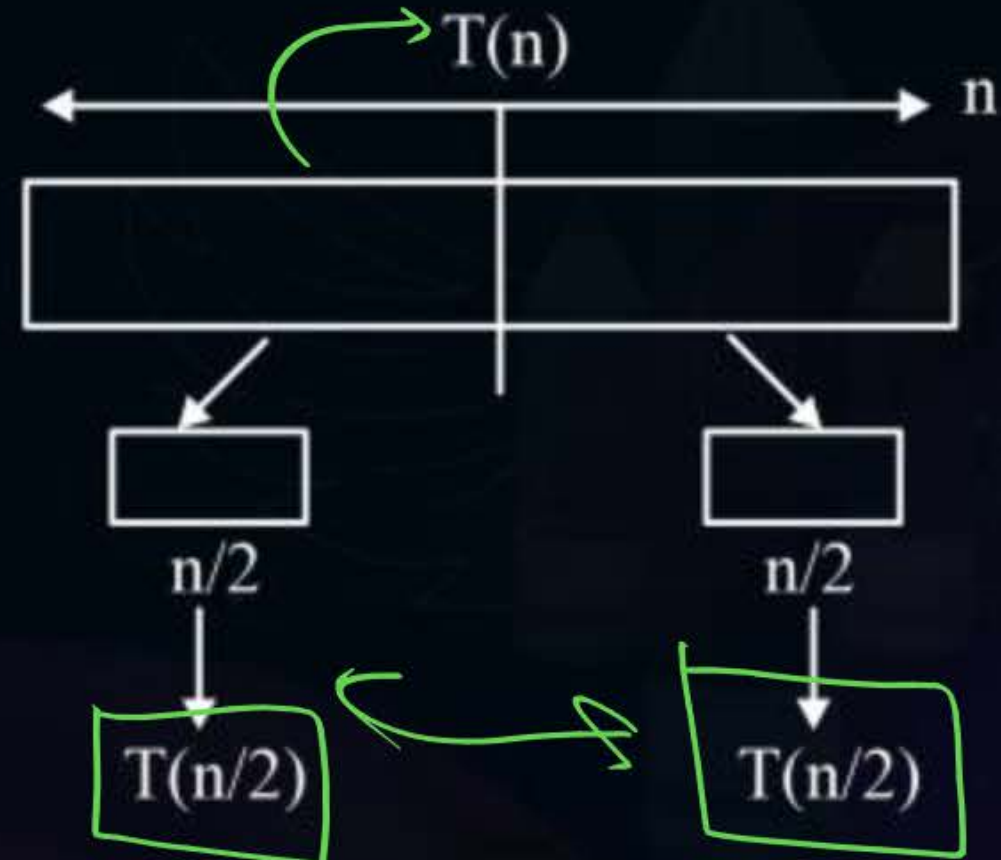
Time complexity for divide & conquer problem:-

Let $T(n) \rightarrow$ original problem P.

$T(n) = g(n)$, if n is small

$= 2 * T(n/2) + F(n)$, if n is large

$$T(n/2) + T(n/2)$$





Topic : Design Strategies

Computing time of DAndC is described by the recurrence relation

$$T(n) \begin{cases} g(n) & \xrightarrow{\text{n small}} \\ \underbrace{T(n_1) + T(n_2) + \dots + T(n_k)} + \underbrace{f(n)} & \xrightarrow{\text{otherwise}} \end{cases}$$



Topic : Design Strategies

Recurrence Relations in Divide & Conquer Problem

- (1) Symmetric
- (2) Asymmetric → Type-1, Type - 2



Topic : Design Strategies

(1) Symmetric D & C

General form: $T(n) = a * T(n/b) + F(n)$

Eg.2. $T(n) = 2T(n/2) + n$

$$T(n) = 3T(n/4) + n^2$$

$$T(n) = 2T(n/3) + n^3$$

no. of
subp

size of each
subp

Master Mtd



Topic : Design Strategies

(2) Asymmetric D & C Type: -1

Eg. $T(n) = T(n/4) + T(3n/4) + F(n)$

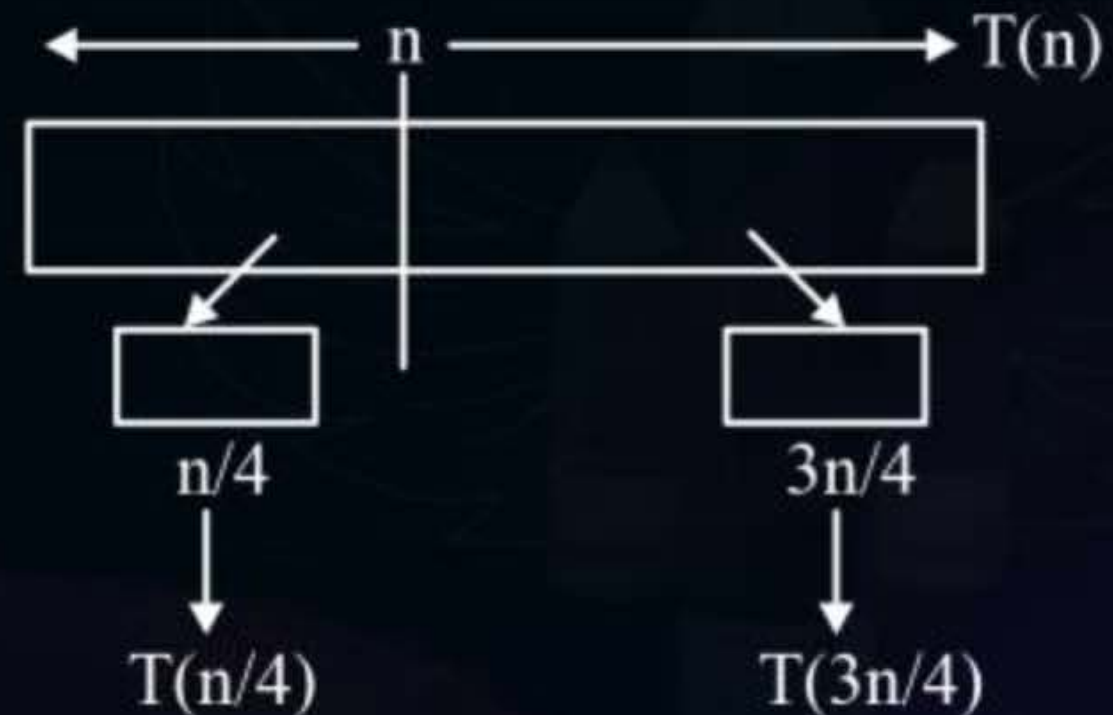
Eg. $T(n) = T(n/3) + T(2n/3) + g(n)$

General form

$$T(n) = T(\alpha n) + T((1-\alpha)n) + g(n)$$

Recursion Tree

Recursion Tree Approach:





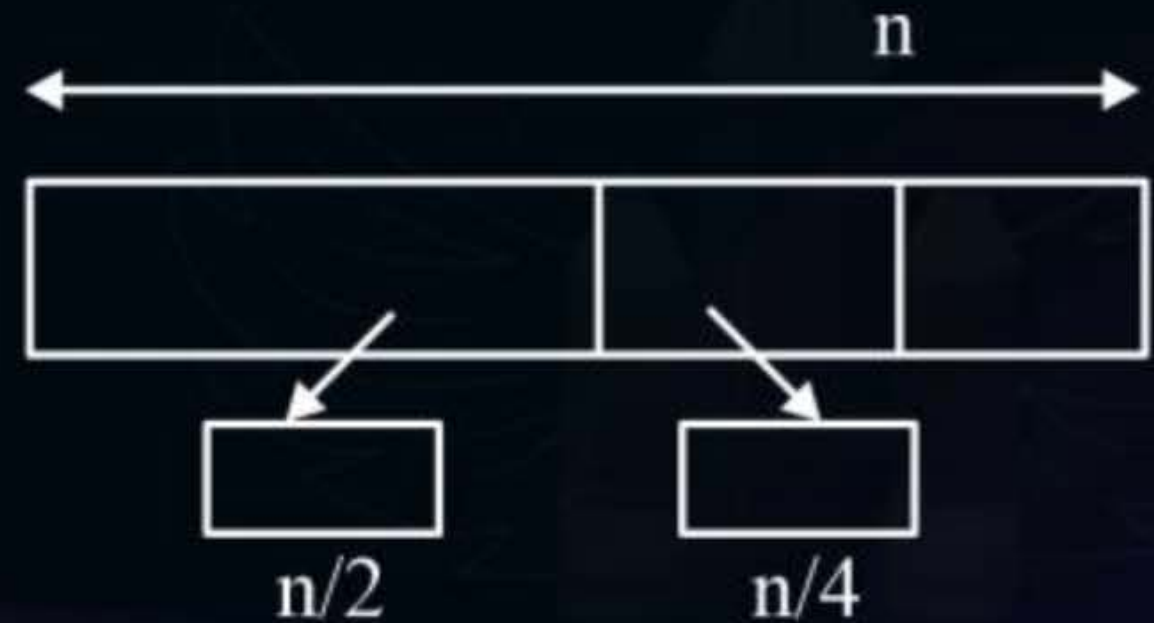
Topic : Design Strategies

(2) Asymmetric D & C Type: -2

Eg. $T(n) = T(n/2) + T(n/4)$

General

$$T(n) = T(\alpha n) + T(\beta n) + T(\delta n) \dots g(n)$$



Recursion Tree Approach:



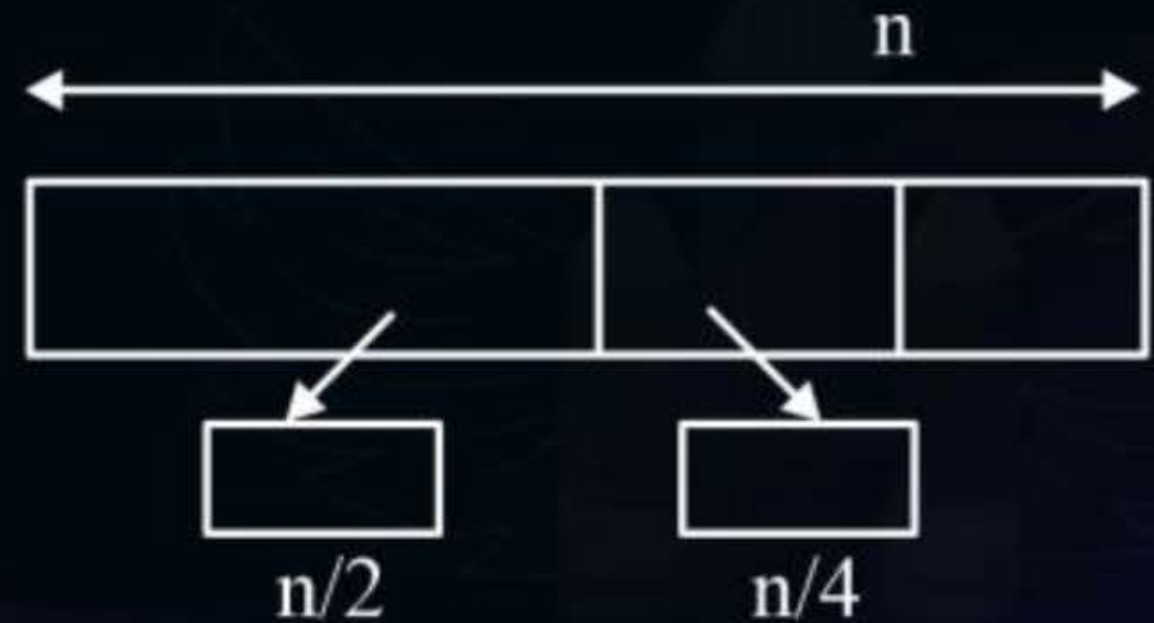
Topic : Design Strategies

(2) Asymmetric D & C Type: -2

Eg. $T(n) = T(n/2) + T(n/4)$

General

$$T(n) = T(\alpha n) + T(\beta n) + T(\delta n) \dots g(n)$$



Recursion Tree Approach:



Topic : Design Strategies

Problem → Divide & Conquer:-

Divide → Mandatory

Conquer → optional

Eg. Binary Search

→ Divide

→ But no need to combine



Topic : Design Strategies

(1) Min – Max Problem



Topic : Design Strategies

(1) Min – Max Problem:

Find min & max element of the array simultaneously



Topic : Design Strategies

Appr I :- Non - D & C Based

Eg.

7	15	-1	-6	17	1	8	59	29	24
1	2	3	4	5	6	7	8	9	10

Algo A] - Min Max (A, n)

```
min = max = A[1]
for(i = 2; i < n; i++)
{
    if(A[i] < min)
    {
        min = A[i]
    }
    if(A[i] > max)
    {
        max = A[i]
    }
}
```

→ $n-1$

→ $n-1$

$$\underline{\underline{\text{Total Comp} = 2 * (n-1)}}$$



Topic : Design Strategies

Algo AI – min-max (A, n)
 min = max = A[1]
 for(i = 2; i < n; i++)
 {
 → if(A[i] < min)
 min = A[i]
 → else if (A[i] > max)
 max = A[i]
 }



Topic : Design Strategies

Time complexity analysis of Aglo2

(1) Best case: input in Decr order

Eg.

10	7	5	2
----	---	---	---

$\Rightarrow (n-1) \text{ Compar}$

(2) Worst case: input in Incr order

Eg.

2	5	7	10
---	---	---	----

$\Rightarrow 2 * (n-1) \text{ Compar}$

$(n-1)$

if

$(n-1)$

else if



Topic : Design Strategies

Bonus: Avg Case:

1 possible Scenario

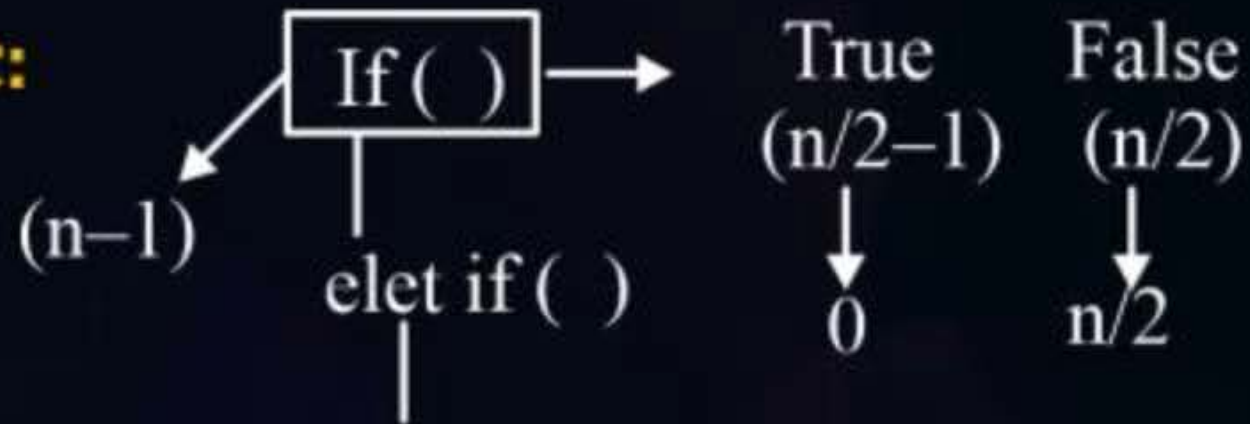
Non-DnC

On an avg, if condition fails 1/2 of the times.

$$\begin{aligned} & \underline{(n-1)} + \frac{n}{2} \\ &= n-1 + \frac{n}{2} = \frac{3n-2}{2} = \boxed{\frac{3n}{2} - 1} \end{aligned}$$

✓ → V. imp Result

Logic:





Topic : Design Strategies

Summary:

V.V.Imp

Exact

$$\underline{\underline{n=10}}$$

Desc:

Asc:

	if	else if	Total Comparision
Best case	$((n-1) + 0)$	$\Rightarrow$	$\underline{(n-1)}$
Wrost case	$(n-1)$	$(n-1) \Rightarrow$	$2*(n-1)$
Avg case	$(n-1)$	$n/2 \Rightarrow$	$\boxed{\frac{3n-1}{2}}$

V.V.Imp

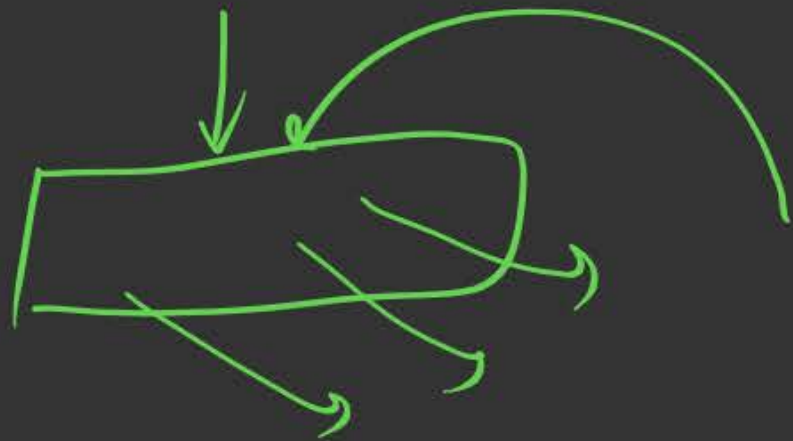
$\rightarrow \underline{\underline{O(n)}}$

One of the Cases

$$\frac{3 \times 10}{2} - 1 = 15 - 1 = 14$$

Soln 2140 :- Insertion Sort

$\Rightarrow \underline{\underline{O(n^2)}}$





2 mins Summary



Topic

Intro to DnC

Topic

Min Max

Topic

Topic

Topic



THANK - YOU